

# Lesson 3 · Drawing Shapes: An Image Buffer and Your First Line

Course: Graphics with C++

Level: Absolute beginner

Est. 60–75 min

In Lesson 1 we colored every pixel with a formula. In Lesson 2 we learned that an image is just a 1D block of bytes in memory, addressed by `index = y * width + x`. Now we combine both ideas to actually **draw**: we'll build a small, reusable **image buffer** in C++, give it a `setPixel` function to color one specific dot, and then draw a straight **line** between any two points — which forces us to face the central awkwardness of graphics: a perfectly straight line has to be built out of square pixels.

Everything here is shown as code plus its real output and lots of pictures. Any math is drawn before it's written.

## 1. A reusable image buffer

Last lesson's gradient printed pixels straight to the screen as it computed them. That's fine for one image, but to *draw* we need to keep all the pixels around in memory, change them in any order, and save them at the end. So we wrap the pixel bytes in a small **struct** — a bundle of related data with functions attached.

### JARGON · STRUCT AND STD::VECTOR

A **struct** groups related variables (and functions) under one name — here, an image's width, height, and its pixels. A **std::vector** is C++'s resizable array; it stores its elements on the *heap* (Lesson 2), which is exactly where a big block of pixels belongs. `std::vector<unsigned char>` is a list of bytes (an **unsigned char** is a one-byte value 0–255 — perfect for a color channel).

#### struct Image

```
int width = 4
int height = 3
vector<byte>
pixels →
```

pixels: one flat 1D array on the heap (row by row)

|       |   |   |   |       |   |   |   |       |   |    |    |
|-------|---|---|---|-------|---|---|---|-------|---|----|----|
| 0     | 1 | 2 | 3 | 4     | 5 | 6 | 7 | 8     | 9 | 10 | 11 |
| row 0 |   |   |   | row 1 |   |   |   | row 2 |   |    |    |

Each of the 12 pixels is 3 bytes (R, G, B):

|   |   |   |
|---|---|---|
| R | G | B |
|---|---|---|

 = pixel 0. So the array is  $12 \times 3 = 36$  bytes.

*The Image struct holds its size plus one flat vector of bytes — the same row-major layout from Lesson 2, now living inside a reusable object.*

Here's the struct:

```
#include <vector>
#include <fstream>

struct Image {
    int width, height;
    std::vector<unsigned char> pixels;    // all bytes start at 0 = black

    Image(int w, int h) : width(w), height(h), pixels(w * h * 3, 0) {}

    void setPixel(int x, int y, int r, int g, int b) {
        if (x < 0 || x ≥ width || y < 0 || y ≥ height) return;    // skip off-image
        int i = (y * width + x) * 3;    // row-major index, x3 for R,G,B
        pixels[i] = r; pixels[i + 1] = g; pixels[i + 2] = b;
    }

    void save(const char* path) {
        std::ofstream f(path);
        f << "P3\n" << width << ' ' << height << "\n255\n";
        for (size_t i = 0; i < pixels.size(); i += 3)
            f << (int)pixels[i] << ' '
              << (int)pixels[i + 1] << ' '
              << (int)pixels[i + 2] << '\n';
    }
};
```

## Reading it piece by piece

- `struct Image { ... };` — defines our new type. Anything of type `Image` carries a width, a height, and a `pixels` vector together.
- `Image(int w, int h) : width(w), height(h), pixels(w * h * 3, 0) {}` — this is the **constructor**, the function that runs when you make an `Image`. The part after the colon fills in the fields: it sets width and height, and creates the `pixels` vector with `w * h * 3` bytes, every one initialized to `0` — a fully black image.

### JARGON · CONSTRUCTOR AND MEMBER FUNCTION

A **constructor** is a special function with the same name as the struct; it sets up a new instance. A **member function** (or *method*) is a function that belongs to the struct and can touch its fields directly — `setPixel` and `save` are member functions of `Image`.

- `setPixel` first checks bounds: if `(x, y)` is outside the image it just returns, so we never write past the array (a common crash). Otherwise it computes `i = (y * width + x) * 3` — the exact row-major formula from Lesson 2, times 3 because each pixel owns 3 bytes — and writes red, green, blue into slots `i`, `i+1`, `i+2`.

- `save` writes the buffer out in the same **PPM** text format from Lesson 1: the `P3` header, then every pixel's three numbers. `std::ofstream` is an **output file stream** — like `std::cout`, but the `<<` goes into a file on disk instead of the terminal. `(int)pixels[i]` **casts** the byte to an `int` so it prints as a number (e.g. `200`) rather than as a raw character.

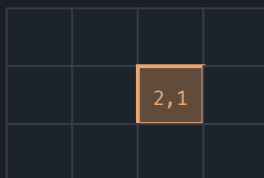
### THE PAYOFF

With just this struct you can create an image, poke any pixel with `setPixel(x, y, r, g, b)` in any order, and `save` it. Everything else in this lesson — and most 2D drawing — is built on top of that one `setPixel`.

## 2. What `setPixel` really does

`setPixel(x, y, ...)` is the bridge between the 2D world we think in and the 1D memory the pixels actually live in. It turns a coordinate pair into an array position:

A 4x3 image — colour pixel (x = 2, y = 1)



$$\begin{aligned} \rightarrow i &= (y \cdot \text{width} + x) \cdot 3 \\ &= (1 \cdot 4 + 2) \cdot 3 = 18 \end{aligned}$$

|    |    |    |
|----|----|----|
| 18 | 19 | 20 |
|----|----|----|

the R, G, B bytes for pixel (2,

`setPixel` converts (x, y) into the byte index `(y · width + x) · 3`. For pixel (2, 1) in a 4-wide image that's byte 18, and its green and blue sit right after at 19 and 20.

## 3. The problem with a line

Now the hard part. Ask for a line from one pixel to another — say from the top-left toward a point 10 columns right and 4 rows down. On paper a line is infinitely thin and perfectly straight. But our image is a **grid of whole pixels**: a pixel is either fully colored or not at all. There's no such thing as "35% of pixel (3, 1)." So the smooth line has to be *approximated* by choosing which whole pixels to light up.



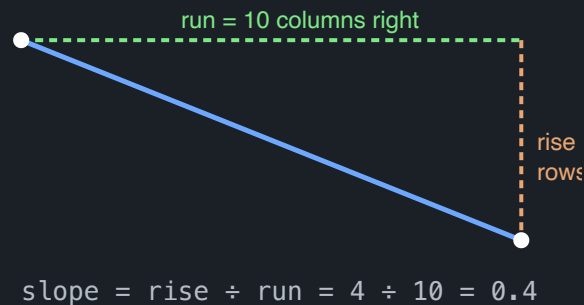
The ideal line slices through many cells only partly — but a

The problem: the true straight line (blue) passes through partial pixels, yet each pixel is all-or-nothing. Which whole pixels do we fill?

## 4. Slope, seen as a picture

The key idea is **slope**, and it's simpler than the word sounds. Slope just answers: "as I take one step to the right, how far up or down does the line go?"

Draw a right triangle under the line. The horizontal side is how far right we travel (the **run**); the vertical side is how far the line moves down (the **rise**). Slope is one divided by the other:



*Slope = rise  $\div$  run. For a line going 10 right and 4 down, slope = 0.4 — meaning each single step right drops the line by 0.4 of a row.*

### THAT'S THE WHOLE SECRET

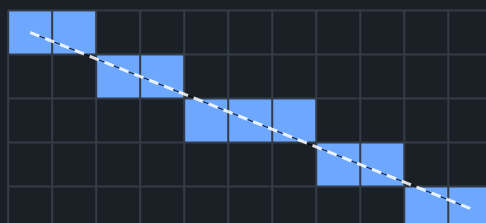
A slope of 0.4 means: every time `x` moves right by 1, the true line moves down by 0.4 of a pixel. After 1 step it's 0.4 down, after 2 steps 0.8 down, after 3 steps 1.2 down, and so on. Drawing the line is just deciding which whole row is closest to that running height at each column.

## 5. Rounding to the nearest row → the staircase

So walk column by column. At column `x`, the true line is `0.4 × x` rows down. Since a pixel must sit on a whole row, we light up the pixel on the **nearest whole row** to that height:

| Column x | True height ( $0.4 \cdot x$ ) | Nearest whole row → pixel y |
|----------|-------------------------------|-----------------------------|
| 0        | 0.0                           | 0                           |
| 1        | 0.4                           | 0                           |
| 2        | 0.8                           | 1                           |
| 3        | 1.2                           | 1                           |
| 4        | 1.6                           | 2                           |
| 5        | 2.0                           | 2                           |
| 6        | 2.4                           | 2                           |
| 7        | 2.8                           | 3                           |
| 8        | 3.2                           | 3                           |
| 9        | 3.6                           | 4                           |
| 10       | 4.0                           | 4                           |

Fill those exact pixels and you get a **staircase** that hugs the ideal line as closely as whole pixels allow:



*The 11 pixels chosen by "nearest whole row" (blue), with the ideal line dashed on top. This staircase is exactly what the code below draws.*

## 6. Bresenham's line algorithm

Rounding  $0.4 \times x$  works, but it uses decimals, and decimals are slower and can drift with rounding errors. **Bresenham's algorithm** does the identical rounding using **only whole numbers** — no decimals at all. The trick: instead of tracking the height as  $0.4$ ,  $0.8$ ,  $1.2$  ..., it keeps a running **error** — how far the line has drifted from the current pixel's row — and only ever adds and subtracts integers. When the drift passes the halfway mark, it steps to the next row and pays the error back down.

## YOU DON'T NEED TO MEMORIZE THE ALGEBRA

The mental model is enough: *march along the longer axis one pixel at a time; keep a tally of drift; when the tally tips past halfway, step over by one row.* The version below also handles lines going in *any* direction (up, down, left, right, steep or shallow) by treating the two axes symmetrically.

```
#include <cstdlib>    // for std::abs

void drawLine(Image& img, int x0, int y0, int x1, int y1, int r, int g, int b) {
    int dx = std::abs(x1 - x0), sx = x0 < x1 ? 1 : -1;    // x distance + dir
    int dy = -std::abs(y1 - y0), sy = y0 < y1 ? 1 : -1;    // y distance + dir
    int err = dx + dy;                                     // the running drift ("error")

    while (true) {
        img.setPixel(x0, y0, r, g, b);                // light the current pixel
        if (x0 == x1 && y0 == y1) break;                // reached the end → stop
        int e2 = 2 * err;
        if (e2 ≥ dy) { err += dy; x0 += sx; }           // step horizontally
        if (e2 ≤ dx) { err += dx; y0 += sy; }           // step vertically
    }
}
```

## Reading it piece by piece

- The parameters are the image (passed by **reference**, `Image&`, so we draw into the real buffer, not a copy), the two endpoints `(x0, y0)` – `(x1, y1)`, and the color.
- `dx` is how far apart the points are horizontally; `sx` is the **step direction**, `+1` if we're going right, `-1` if left. `dy` and `sy` are the same vertically (`dy` is stored negative, a neat trick that lets the two axes share one formula). The `? :` is the **ternary operator** — `condition ? a : b` means "use `a` if the condition is true, else `b`."
- `err` is the integer stand-in for that fractional drift from §5.
- Inside the loop we colour the current pixel, stop if we've arrived, then the two `if`'s decide whether this step moves us across, down, or both. Each move adjusts `err` to keep the running tally honest. Because everything is `int`, there's not a single decimal in sight — that's Bresenham's whole point.

## JARGON · REFERENCE PARAMETER AND WHILE(TRUE)

An `&` on a parameter ( `Image& img` ) passes the actual object by **reference** — changes inside the function affect the caller's image (and nothing gets copied, which matters for big buffers).

`while (true)` is a loop with no built-in stop; it runs until a `break` statement jumps out — here, when we reach the end pixel.

## 7. Drawing something with it

Now the fun part. Here's a `main` that makes a 256×256 image and draws 24 colored lines radiating from the center — a starburst — then saves it:

```
#include <cmath>    // for std::cos, std::sin

int main() {
    Image img(256, 256);
    int cx = 128, cy = 128;           // center of the image
    int cols[6][3] = {{110,168,254},{126,231,135},{232,166,114},
                      {255,123,114},{200,150,255},{121,192,255}};
    for (int k = 0; k < 24; ++k) {
        double a = k * (2 * 3.14159265 / 24); // one of 24 angles on a circle
        int x1 = cx + (int)(118 * std::cos(a)); // point on radius-118 circle
        int y1 = cy + (int)(118 * std::sin(a));
        int* c = cols[k % 6];                 // cycle the 6 colors
        drawLine(img, cx, cy, x1, y1, c[0], c[1], c[2]);
    }
    img.save("lines.ppm");
    return 0;
}
```

Each pass around the loop picks an **angle** `a` spread evenly around a full turn, finds the point that far around a circle (using `cos` and `sin` — we'll properly meet those when we do rotation later; here just trust that they place a point on a circle), and draws a line from the center out to it. Compiled and run, `drawLine` fires in every direction, so we're exercising all those step-direction cases at once. The saved image looks like this:



The real output of the program above (converted from `lines.ppm` to PNG). 24 straight lines, every one built pixel-by-pixel by the same tiny `drawLine`.

## WHAT YOU CAN NOW BUILD

A line is the first real building block. Three lines make a triangle outline; many short lines make curves; a grid of lines makes graphs and wireframes. And `setPixel` + `drawLine` are enough to draw the outline of almost any 2D shape. Filling those shapes with solid color is the next big idea (rasterization), which we'll get to with triangles.

## 8. What you learned

- An **image buffer** is a struct holding `width`, `height`, and a flat `std::vector` of bytes (heap memory) — a reusable object you can draw into and save.
- `setPixel(x, y, r, g, b)` is the bridge from 2D coordinates to 1D memory: it writes 3 bytes at `(y*width + x)*3`, with a bounds check to stay safe.
- A straight line can't be drawn exactly on a pixel grid — pixels are **whole, all-or-nothing** — so we approximate it with a **staircase** of the nearest pixels.
- **Slope = rise ÷ run** tells you how far the line moves per step; picking the **nearest whole row** to the running height gives the staircase.
- **Bresenham's algorithm** produces that same staircase using **only integers** (a running "error" tally instead of decimals), and handles lines in every direction.
- New vocabulary: struct, member function, constructor, `std::vector`, `unsigned char` / byte, `std::ofstream`, cast `(int)`, reference parameter `&`, ternary `? :`, `while(true)` / `break`, slope, rise, run, Bresenham, staircase (aliasing).

Tell me which section to expand or slow down and I'll revise this lesson and rebuild the PDF.